M1 Informatique - HAI818I - Modularité et Réutilisation

Année 2025-2026, Session 1, mai 2026.

C. Dony, I. Mougenot, C. Nebut, F. Vargas-Rojas

Durée : 2h00. **Documents non autorisés**. La précision et la concision des réponses sont évaluées positivement. La partie I porte sur les 4 premières séances de cours (C. Dony, C. Nebut) et la partie II sur les suivantes (I. Mougenot, F. Vargas-Rojas). Les parties sont indépendantes. Chaque partie vaut 10 points.
Merci de rédiger les réponses aux parties 1 et 2 de façon séparée (feuilles différentes).

PARTIE I (10 points et 1 heure de temps estimé)

A Partie I-A (environ 4 points)

1. Donnez, par un texte très court, l'algorithme d'envoi de message (également nommé appel de méthode). En d'autres termes, que se passe-t-il quand la machine virtuelle Java (par exemple) exécute l'instruction `o.m()` ?
2. Considérons à nouveau l'instruction d'envoi de message `o.m()`. Qui est l'envoyeur du message ? Qui est le receveur ?
3. En programmation, donnez une différence essentielle entre appel de méthode et appel de fonction ? Quel impact sur la réutilisation ?
4. Pensez vous qu'on puisse faire une analogie entre l'envoi de SMS (ou de mails) entre humains et l'envoi de message en programmation ?

B Partie I-B (environ 6 points)

1. **Compréhension des mécanismes de proxies et de tissage**

   Expliquez en quelques phrases la/les différence(s) fondamentale(s) (principe, moment d'application, ...) entre l'utilisation de proxies (tels que créés par ByteBuddy par exemple) ou de tissage (comme réalisé par AspectJ).

2. **Capacité d'interception**

```
1  public class Order {
2      @Sync
3      public User customer;

5      public void setCustomer(User c) {
6          this.customer = c; //(A) }
7  ...
8  public static void main(String[] args){
9      Order order =new Order();
10     User user=new User();
11     order.setCustomer(user); //(B)
12     order.customer=user; //(C) }

14 // on suppose qu'existent la classe User et l'annotation @Sync ...
```

Listing 1 –

   Etant donné le code ci-dessus, indiquez pour chacune des lignes A, B et C, si l'accès au client (customer) peut être intercepté dans le cas de l'utilisation d'un proxy pour Order d'une part, et dans le cas de l'utilisation d'un aspect de type AspectJ d'autre part.

3. **Fabrique d'objets**

   Lors du TP réalisé autour des aspects et des proxies pour la mise en place de synchronisation automatique entre attributs (représentant des extrémités d'association bidirectionnelle), vous avez mis en place la synchronisation avec d'une part des proxies, et d'autre part aspectJ. Dans un cas vous avez eu recours à une fabrique d'objets, dans l'autre cas non. Expliquez cette différence.

PARTIE II (10 points et 1 heure de temps estimée)

Merci de rédiger les réponses aux parties I et II de façon séparée (feuilles différentes).

Les exemples de code sont repris directement de l'exemple applicatif exploité en TP (les campus de l'Université)

A Mapping Objet-Relationnel (3 points)

1. Expliquer les difficultés rencontrées lors de la mise en correspondance entre un modèle objet et un schéma relationnel (différence de paradigme). Ces difficultés sont bien identifiées et regroupées sous une notion que vous nommerez. Vous expliquerez les moyens mis en œuvre pour contourner ces difficultés.

2. Un extrait de la classe Campus mettant en évidence le lien entre le campus et ses bâtiments d'enseignement est présenté.

```
@Entity
public class Campus {

@OneToMany(fetch = FetchType.EAGER, mappedBy="campus", cascade = CascadeType.ALL)
private List<Batiment> batiments = new ArrayList<Batiment>();
}
```

Listing 1: dans la classe Campus

- Vous direz à quoi correspond l'annotation @OneToMany
- Si un campus est détruit, est ce que les bâtiments qui en dépendent le sont aussi ?
- Est ce que la stratégie FetchType.EAGER vous semble une bonne solution et pourquoi ?

B Composants et injection de dépendances (5 points)

1. Un extrait de code associé au composant CampusRepository est fourni

```
@Repository
public interface CampusRepository extends JpaRepository<Campus, String> {

   @Query("SELECT DISTINCT c FROM Campus c JOIN FETCH c.batiments")
   List<Campus> findCampus();
   }
```

Listing 2: dans le composant CampusRepository

   (a) *expliquer les deux types génériques retrouvés au niveau de l'interface JpaRepository*

   (b) expliquer l'intérêt du JOIN FETCH et du DISTINCT dans la requête JPQL qui sous-tend la méthode findCampus()

2. un extrait de la classe CampusService est fourni, et vous répondrez à un ensemble de questions correspondant à cet extrait :

```
@Service
@Primary
public class CampusService implements ICampusService {

   private final CampusRepository cr;

   @Autowired
   public CampusService(CampusRepository cr) {
      this.cr = cr;
   }

   @Transactional(rollbackFor = Exception.class, timeout=4)
   public void updateCampus(String ville, String nomC) {
         cr.modifieVilleCampus(ville, nomC);
   }
```

Listing 3: dans un composant Service

   (a) pourquoi est t'il important que la classe CampusService implémente l'interface ICampusService et quel est l'intérêt de l'annotation @Primary ?

(b) où se serait situé le couplage fort en l'absence de la mise en œuvre de l'injection de dépendance ?

(c) expliquer le rôle de l'annotation @Transactional pour la méthode updateCampus, et donner la signification des attributs rollbackFor et timeout et de leurs valeurs

(d) quels sont les principes orthogonaux mis en jeu lorsqu'une méthode décorée par l'annotation @Transactional est appelée ?

C Spring MVC (2 points)

1. Expliquer, en trois ou quatre lignes, les intérêts du pattern "Front Controller" dans Spring MVC. Vous pouvez vous aider d'un schéma explicatif.